

< 200 CONCRETE OPEN PROBLEMS IN MECHANISTIC INTERPRETABILITY >

200 COP in MI: Looking for Circuits in the Wild

by **Neel Nanda** 29th Dec 2022

This is the third post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. [Start here](#)[°], then read in any order. If you want to learn the basics before you think about open problems, check out [my post on getting started](#). Look up jargon in [my Mechanistic Interpretability Explainer](#)

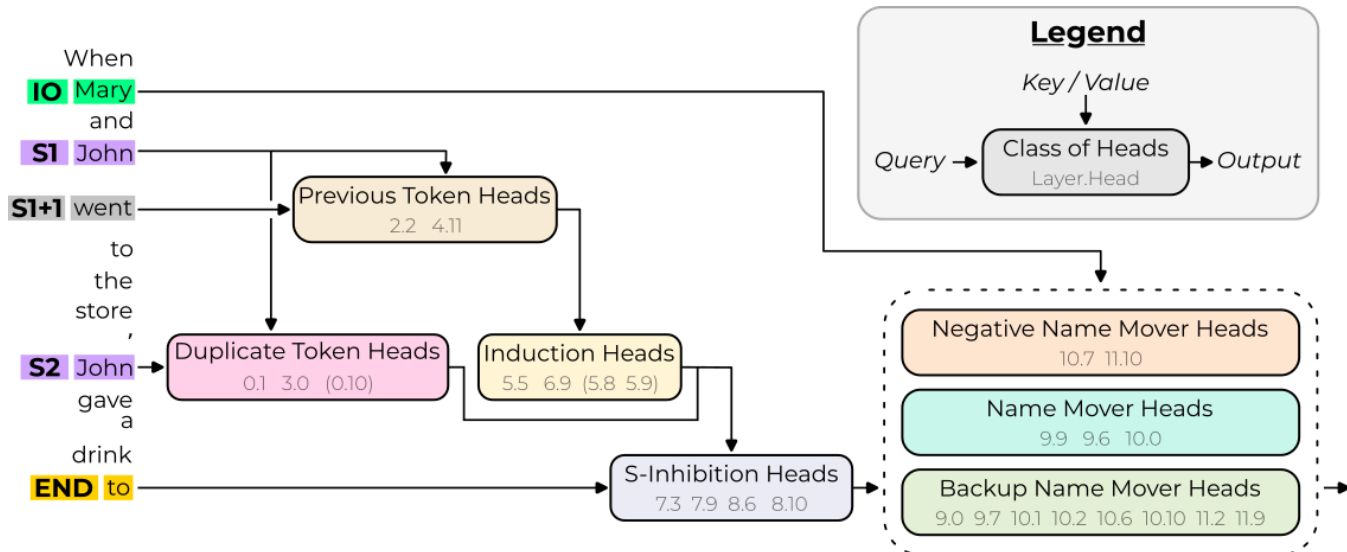
Motivation

Motivating paper: [Interpretability In The Wild](#)

Our ultimate goal is to be able to reverse engineer real, frontier language models. And the next step up from toy language models is to look for circuits in the wild. That is, taking a real model that was *not* designed to be tractable to interpret (albeit much smaller than GPT-3), take some specific capability it has, and try to reverse engineer how it does it. I think there's a lot of exciting low-hanging fruit here, and that enough groundwork has been laid to be accessible to newcomers to the field! In particular, I think there's a lot of potential to better uncover the underlying principles of models, and to leverage this to build better and more scalable interpretability techniques. We currently have maybe *three* published examples of well-understood circuits in language models - I want to have at least 20!

This section is heavily inspired by Redwood Research's recent paper: [Interpretability In The Wild](#). They analyse how GPT-2 Small solves the grammatical task of [Indirect Object Identification](#): e.g. knowing that "John and Mary went to the store, then John handed a

bottle of milk to” is followed by Mary, not John. And in a heroic feat of reverse engineering, they rigorously deciphered the algorithm. In broad strokes, the model first identifies which names are repeated, inhibits any repeated names, and then predicts that the name which is *not* inhibited comes next. The full circuit consists of 26 heads, sorted into 7 distinct groups, as shown below - [see more details in my MI explainer](#). A reasonable objection is that this is a cherry-picked task on a tiny (100M parameter) model, so why should we care about it? But I’ve learned a lot from this work!



It’s helped uncover some underlying principles of networks. Notably that the model has **built in redundancy**: if the name mover heads (which attend to the correct name) are **knocked-out**, then there are backup heads which take over and do the task instead! And that the model has learned to **repurpose existing functionality**: the model has a component (**induction heads**) for the more complex task of continuing repeated subsequences, but here they *also* get re-purposed to detect whether a name is repeated. It would have been much harder to understand what was going on here without the prior work of reverse-engineering induction circuits! And there’s many open questions building on the work that could clarify other principles like **universality**: when another model learns this task, does it learn the same circuit?

Further, it’s helped to build out a toolkit of techniques to rigorously reverse engineer models. In the process of understanding this circuit, they refined the technique of **activation patching** into **more sophisticated approaches** such as path patching (and later **causal scrubbing**). And this has helped lay the foundations for developing future techniques! There are many interpretability techniques that are more scalable but less mechanistic, like **probing**. If we have some well understood circuits, and can see how well these techniques work in *those*

settings (and what they miss!), this gives grounding to find the *good* techniques and how to understand their results.

Finally, I think that enough groundwork has been laid that finding circuits in the wild is **tractable**, even by people new to the field! There are sufficient techniques and demos to build off of that you can make significant progress without much background. I'm excited to see what more we can learn!

Resources

- **Demo:** A tutorial for my [TransformerLens library](#) where I [demonstrate basic techniques for initial exploratory analysis](#), doing a broad strokes analysis of the Indirect Object Identification circuit.
 - I demonstrate how to use [direct logit attribution](#) and [activation patching](#)
 - **Start by copying this notebook and using it as a starting point**
- My [Mechanistic Interpretability Explainer](#), especially the sections on the [Indirect Object Identification Circuit](#) and on [MI Techniques](#).
- The [interpretability in the wild](#) paper
 - [Twitter thread summary on the paper](#)
 - [A \(2 part\) video walkthrough of Interpretability in the Wild](#) where I interview the authors
 - The description contains a link to a lot of other resources!
 - Their [codebase](#) & [accompanying colab](#) (based on an early version of TransformerLens)
- TransformerLens lets you load in many GPT-2 style models ([documented here](#)), and the same code will work with any of them (just change the string in ``from_pretrained``). This makes it easy to find the best model to study and to look at how circuits differ between models!
 - It also contains [some models I trained](#) with [SoLU activations](#) on the MLP layers, up to GPT-2 Medium size. These make neurons more interpretable and are likely easier to study for tasks involving MLP layers!
- If you're new to transformers, check out the transformer intro resources in [my getting started guide](#)° - it *really* helps to have a good intuitions for transformers, and to have coded one yourself

Tips

- **Recommended workflow:**

1. Identify a behaviour (like Indirect Object Identification) in a model that you want to understand
 2. Try to *really* understand the behaviour as a black box. Feed in a lot of inputs with many variations and see how the model's behaviour changes. What does it take to break the model's performance? Can you confuse it or trip it up?
 3. Now, approach the task as a scientist. Form hypotheses about what the model is doing - how *could* a transformer implement an algorithm for this? And then run experiments to support and to falsify these hypotheses. Aim to iterate fast, be exploratory, and get fast feedback.
 - a. Copy my [exploratory analysis tutorial](#) and run the model through it on some good example prompts, and try to **localise** what's going on - which parts of the model seem most important for the task?
 4. Regularly red-team yourself and look for what you're missing - if there's a boring explanation for what's going on, or a flaw in your techniques, what could it be? How could you falsify your hypothesis? Generating convoluted explanations for simple phenomena is one of the biggest beginner mistakes in MI!
 5. Once you have some handle on what's going on, try to scale up and be more rigorous - look at many more prompts, use more refined techniques like path patching and causal scrubbing, try to actually reverse engineer the weights, etc. At this stage you'll likely want to be thinking about more ad-hoc techniques that make sense for your specific problem.
- All of the [tips in my toy language model piece](#)° apply here! In particular the thoughts on problem choice - in brief, the most tractable problems are likely to be those that are algorithmic-ish, where you want to understand how the model knows to output a single token, where there are two natural answers (one correct and the other incorrect) such that you can study [the difference in logits between the two](#), and where the structure comes up a *lot* in the training data.
 - Indirect Object Identification is a great example - it's easy to write an algorithm to solve it, we can study single token names, John vs Mary are the two natural answers, and it's a common grammatical structure so it comes up a lot in the training data - and so the model is incentivised to spend a lot of parameters being good at it!
 - Obviously, research on less cherry-picked problems is what we actually want. But I recommend *starting* with a problem selected for being tractable, learning skills,

and trying harder ones later. Don't bite off more than you can chew! The problems below have a wide range in difficulty.

- Look for the smallest model size capable of doing a task, and try to interpret that first. This will be more tractable, and should make later analysing the larger models much easier. TransformerLens code will work for any model if you just change the name in ``from_pretrained`` so this is easy to explore.
 - In particular, check whether a toy language model can do that, and try starting there if possible.
- Transformers have two types of layers - [attention layers](#) (made up of [attention heads](#)) and MLP layers (made up of neurons). Intuitively, attention is about routing information between different positions in the prompt, and MLPs are about processing information *at* a position in the prompt. When forming hypotheses or analysing results, remember to keep this in mind!
 - It makes sense that the IOI task is done by a circuit of (mostly) heads: it's fundamentally about moving information around the model (copying the correct name and *not* copying the repeated name!)
 - MLPs are likely to be necessary for anything involving non-linear reasoning.
 - For example, a trigram like “ice cream -> sundae” needs “previous token == ice” AND “current token == cream” (“ice rink” and “clotted cream” should *not* imply sundae!). AND is non-linear, and so this is likely done with MLPs
 - Also, we're much more confused about how to reverse engineer MLPs, since [polysemanticity](#) and [superposition](#) remain big mysteries! So expect these tasks to be harder.
- It's useful to keep a mental distinction between **exploratory** and **rigorous** techniques. There's lots of techniques to get information about a model's internals, which span the spectrum from simple but unreliable (eg looking at attention patterns) to literally [replacing the parameters with weights that you've hand-coded](#).
 - My personal workflow is to start with a range of exploratory techniques and iterate fast to narrow down what's going on, while remaining aware of the weaknesses in my approaches and trying to red-team. And then transitioning to more involved and rigorous techniques as I form more confident hypotheses.
 - The important thing is not the exact workflow, but keeping track of the trade-off between speed and rigour, and remaining skeptical and self-aware of the standards of your analysis.

- GPT-2 Small's performance is ruined if you [ablate](#) the first MLP layer. This probably isn't relevant to your specific task, and I wouldn't focus on it.
- *Overly technical tip:* A natural thing is to want to study larger models (eg GPT-NeoX, with 20B parameters). Large models can do fancier and more exciting things, like arithmetic and few-shot learning! I'm excited to see work here, but I recommend against this as a first project. Large models are a *massive* pain, and will be slower, messier, and require more involved and expensive compute infrastructure.
 - A core constraint here is GPU memory - the model's weights and activations need to be stored on the GPU, most GPUs have ≤ 16 GB in memory, and more expensive ones have ≤ 40 GB (the very best have 80GB). And life is much easier with models on a single GPU.
 - Rule of thumb: a free Colab notebook can roughly run GPT-2 XL (1.5B), a decent GPU can run GPT-J (6B) in low precision, for NeoX and up you probably want multiple GPUs or a really high end one.
 - Though TransformerLens will hopefully soon support multi-GPU models!
 - Convert the model to low precision (float16 or bfloat16) so it is faster and takes up half as much space. This will introduce *some* error, but probably not enough to change things dramatically.
 - Put ``torch.set_grad_enabled(False)`` at the top of your notebook to turn off automatic differentiation, this can easily clog up your GPU memory otherwise.

Problems

[This spreadsheet](#) lists each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or reach out to other people on there! (thanks to Jay Bailey for making it)

- Circuits in natural language
 - **B* 2.1** - Look for the induction heads in GPT-2 Small that work with [pointer arithmetic](#). Can you reverse engineer the weights?
 - **B* 2.2** - Continuing common sequences (*not* sequences that have already occurred in the prompt, but sequences that are common in natural language)
 - Eg "1 2 3 4" -> "5", "1, 2, 3, 4,"->"5", "Monday\nTuesday\n->"Wednesday", "I, II, III, IV," -> "V", etc
 - **B* 2.3** - A harder example would be numbers at the start of lines (of arbitrary length), eg "1. Blah blah blah \n2. Blah blah blah\n->"3.". Feels like

it must be doing something induction-y!

- **B* 2.4** - 3 letter acronyms, like “The Acrobatic Circus Group (ACG) and the Ringmaster Friendship Union (“ -> RFU (GPT-2 Small is pretty good at this!)
- **B* 2.5** - Converting names to emails, like “Katy Johnson <->”katy_johnson” (GPT-2 Small is pretty good at this!)
 - C An extension task is e.g. constructing an email from a snippet like the following:
Name: Jess Smith
Email: last name dot first name k @ gmail
- **C* 2.6** - Interpret factual recall. You can heavily crib off of the [ROME paper’s work with causal tracing here](#), but how much more specific can you get? Can you find specific heads, or ideally specific neurons?
 - I recommend starting with my largest [SoLU model](#) (`solu-12l`), running it on the facts in the paper’s CounterFact dataset (I made [a stripped down version here](#)) and looking for facts the model can do competently. The task seems to involve MLPs, so I expect it to be easier with a SoLU model, which make MLP neurons more interpretable.
 - Be careful to check that the model is *actually* doing factual recall. Eg GPT-2 knows that “Danielle Darieux’s mother tongue is” -> “French”, but it’s easy to just learn correlations with French sounding names (eg a skip trigram like “Darieux...is” -> “French” is easy to learn, but not really factual recall)
 - I’d try a range of facts, and look for one that seems more tractable and easy to localise.
- **B* 2.7** - Learning that words after full stops are capital letters
 - I predict that you should be able to find a “begins with a capital letter” direction in the residual stream that the model uses here!
- **B-C* 2.8** - Counting objects described in text.
 - E.g.: I picked up an apple, a pear, and an orange. I was holding *three* fruits.
- **C* 2.9** - Interpreting memorisation. E.g., there are times when GPT-2 knows surprising facts like [people’s contact information](#). How does that happen?
- **B 2.10** - Reverse engineer an induction head in a non-toy model, eg GPT-2 Small
 - [Here’s a heatmap of the induction heads in many models](#)
- **B 2.11** - Choosing the right pronouns (e.g. he vs she vs it vs they)

- A good setup is a rhetorical question (so it doesn't spoil the answer!) like "Lina is a great friend, isn't" (h/t Marius Hobbhahn)
 - A-C 2.12 - Choose your own adventure! Try finding behaviours of your own. I recommend just feeding in a bunch of text and looking for interesting patterns that the model can pick up on.
- Circuits in code models (Note: GPT-2 was not trained on code. Try out GPT-Neo, Pythia or my SoLU models, which were)
 - **B* 2.13** - Closing brackets
 - Bonus: Tracking the right kinds of brackets - [, (, {, < etc
 - **B* 2.14** - Closing HTML tags
 - **C* 2.15** - Methods depend on object type - eg you see x.append if it's a list vs x.update if it's a dictionary (in Python)
 - **A-C* 2.16** - Choose your own adventure! Look for interesting patterns in how the model behaves on code, and try to reverse engineer something. I expect it to be easiest on algorithmic flavoured tasks.
- Extensions to the IOI paper (read [my explainer on it](#) to get context. Thanks to Alex Variengien, an author, for contributing many of these)
 - **A* 2.17** - Understand IOI in the [Stanford mistral models](#) - these are GPT-2 Small replications trained on 5 different random seeds, does the same circuit arise? (You should be able to near exactly copy Redwood's code for this)
 - **A* 2.18** - Do earlier heads in the circuit (duplicate token, induction & S-Inhibition) have backup style behaviour? If we ablate them, how much does this damage performance? Is it as we expect, or do other things step in to compensate?
 - **B* 2.19** - See if you can figure out what's going on here - is there a general pattern for backup-ness?
 - **A* 2.20** - Can we reverse engineer how the duplicate token heads work deeply? In particular, how does the QK circuit know to look for *copies* of the current token, without activating on non-duplicate inputs because the current token is always a copy of itself?
 - I recommend studying this on random tokens with some repeats. This should be cleaner than the IOI task because less other stuff is going on.
 - **B* 2.21** - Understand IOI in GPT-Neo: it's a same size model but seems to do IOI via composition of MLPs

- **C* 2.22** - What is the role of Negative/ Backup/ regular Name Movers Heads outside IOI? Can we find examples on which Negative Name Movers contribute positively to the next-token prediction?
- **C* 2.23** - What are the conditions for the compensation mechanisms where ablating a name mover doesn't reduce performance much to occur? (ie backup name mover heads activating and negative name movers reducing in magnitude) Is it due to dropout?
 - **B* 2.24** - A good place to start is looking at GPT-Neo, which wasn't trained with dropout. Does it have backup heads? What happens when you ablate name movers?
 - *Arthur Conmy is working on this - feel free to reach out to arthur@rdwrs.com*
- B 2.25 - Reverse engineering understanding 4.11, (a really sharp previous token heads) at the parameter level.
 - The attention pattern is almost perfectly off-diagonal, so this seems likely to be pretty tractable! But it might be using some parts
- C 2.25 - MLP layers (beyond the first one) seem to matter somewhat for the IOI task. What's up with this?
 - My personal guess is that the MLPs are doing some generic behaviour, like boosting or suppressing certain directions in the residual stream, that aren't IOI specific
- C 2.26 - Understanding what's happening in the adversarial examples: most notably S-Inhibition Head attention pattern (hard).
- Things that confuse me in models
 - **B-C* 2.27** - Models have [a lot of induction heads](#). Why? Can you figure out how these heads specialise and why the model needs so many?
 - **B* 2.28** - GPT-2 Small's performance is ruined if you ablate MLP0 (the first MLP layer). Why?
 - I hypothesise that this is because GPT-2 Small [ties embedding weights](#) (uses the same matrix to embed and unembed tokens). In my opinion, this is unprincipled because the model wants to do different things when it embeds and unembeds tokens. So it uses the MLP0 as an "extended unembedding". And that the outputs are mostly just functions of the token at that position (and not earlier tokens!), and are treated by later layers as "this is the input token".

- But I haven't checked this! I'd start by studying how much Attn0 attends to earlier tokens and how much the outputs of MLP0 depend on prior tokens, or whether there's some subspace that just depends on the current token.
 - I'd also study the IOI circuit and try a range of prompts with the same names - can you average the MLP0 outputs to find a "John direction"? What happens if you replace the MLP0 output with this "John direction"?
 - **B-C* 2.29** - Can we find evidence of the [residual stream as shared bandwidth hypothesis](#)?
 - **B* 2.30** - In particular, the idea that the model dedicates parameters to memory management, and cleaning up memory once it's used. Eg are there neurons with high negative cosine sim? (so the output erases the input feature) Do these correspond to cleaning up specific features?
 - **B* 2.31** - What happens to the memory in an induction circuit?
 - Concrete Q: The previous token head's output is used by the induction head via K-Composition. What else happens to this message? Is it gone by the unembed? Do other heads/MLPs use it? What happens if you use path patching to delete it from the input of some other components?
- Studying larger models
 - **B-C* 2.32** - [GPT-J contains translation heads](#). Can you interpret how they work and what they do? I recommend starting with the specific prompt in that link, and trying eg activation patching as you change specific words. I think getting some results will be easy, but that there'll be enough messy details that truly understanding this will be hard.
 - **C* 2.33** - Try to find and reverse engineer fancier induction heads, eg [pattern matching heads](#) - I'd look in GPT-J or GPT-NeoX
 - **C-D* 2.34** - What's up with few-shot learning? How does it work?
 - I recommend starting with a concrete few-shot problem that GPT-J or GPT-NeoX can do
 - **C* 2.35** - How does addition work?
 - I'd focus on 2 digit addition - GPT-J gets 15% accuracy and GPT-NeoX gets 50%, which is good enough that something real should be happening! ([See Figure 9](#)) Smaller models aren't going to cut it here
 - You want to try several input formats here and look for the nicest. Tokenization can make this messy - numbers are not tokenized with

consistent numbers of digits per token!

- [My grokking work](#) is hopefully relevant here - in the two digit tokens case, I would look for directions in the residual stream corresponding to $\cos((a + b + x)\omega)$, $\sin((a + b + x)\omega)$, where $\omega = \frac{2\pi k}{100}$ and a, b are the two input digits at that position and x is a possible carry.
- **C* 2.36** - What's up with [Tim Dettmer's emergent features](#) in the residual stream stuff? (Turns out that some residual stream coordinates are systematically way bigger than others) Do these map to anything interpretable? What happens if we do max activating dataset examples on these?
 - One hypothesis is that they're being used to control the layernorm scaling down function, and that the layernorm scaling weights (the elementwise multiply after scaling down) move these features back to the normal scale
 - See [this great post by Anthropic](#) arguing that Adam's lack of basis invariance is the cause of these

Previous:

200 COP in MI: The Case for Analysing Toy Language Models

2 comments 40 karma

Next:

200 COP in MI: Interpreting Algorithmic Problems

No comments 33 karma

[Log in to save where you left off](#)

Mentioned in

- 41 200 Concrete Open Problems in Mechanistic Interpretability: Introduction
- 15 Othello-GPT: Future Work I Am Excited About
- 14 Mechanistic Interpretability Quickstart Guide
- 11 200 COP in MI: Studying Learned Features in Language Models
- 11 200 COP in MI: Analysing Training Dynamics

3 comments, sorted by top scoring

[-] **LawrenceC** 3y ▼ 2 ▲ ✕ 1 ✓

- **C*** What is the role of Negative/ Backup/ regular Name Movers Heads outside IOI? Can we find examples on which Negative Name Movers contribute positively to the next-token

prediction?

So, it turns out that negative prediction heads appear ~everywhere! For example, Noa Nabeshima found them on ResNeXts trained on ImageNet: there seem to be heads that significantly reduce the probability of certain outputs. IIRC the explanation we settled on was calibration; ablating these heads seemed to increase log loss via overconfident predictions on borderline cases?

[–] **Noosphere89** 3y ▼ 2 ▲ ✕ 0 ✓

Further, it's helped to build out a toolkit of techniques to rigorously reverse engineer models. In the process of understanding this circuit, they refined the technique of activation patching into more sophisticated approaches such as path patching (and later causal scrubbing). And this has helped lay the foundations for developing future techniques! There are many interpretability techniques that are more scalable but less mechanistic, like probing. Having some

See a Twitter thread of some brief explorations I and Alex Silverstein did on this

I think you cut yourself off there both times.

[–] **Neel Nanda** 3y ▼ 1 ▲ ✕ 0 ✓

Lol thanks. Fixed

[–] **Noosphere89** 3y ▼ 1 ▲ ✕ 0 ✓

You're welcome, though did you miss a period here or did you want to write more?

See a Twitter thread of some brief explorations I and Alex Silverstein did on this

[–] **Neel Nanda** 3y ▼ 1 ▲ ✕ 0 ✓

Missed a period (I'm impressed I didn't miss more tbh, I find it hard to remember that you're supposed to have them at the end of paragraphs)

Moderation Log

More from Neel Nanda



[View more](#)

Curated and popular this week

